

mr460-week5-clustering

Table of contents

1	Step 1 — Unit of analysis	2
1.1	Skew handling (practical)	4
1.2	4A — K-means (required)	6
1.2.1	Choosing k (avoid arbitrary decisions)	6
1.3	4B — Inspect cluster centres (interpretation-friendly)	9
1.4	4C — Visualise clusters with PCA (2D view)	9
1.5	Are clusters balanced?	11
1.6	Sensitivity + limitations	11

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(lubridate)
library(here)
```

here() starts at /Users/trodgers/Documents/GitHub/dmnn-2526-sem2-demo

```
# clustering helpers
library(cluster) # silhouette()
set.seed(42)
```

1 Step 1 — Unit of analysis

Unit of analysis: customers (one row per customer).

Why this unit makes sense for segmentation:

Customer segmentation aims to group people with similar purchasing behaviour so we can describe “types” of shoppers (e.g., frequent small-basket buyers vs. rare high spenders). The Online Retail data is transactional (many rows per customer), so clustering requires us to **aggregate** transactions into a compact customer “behaviour profile” (features) before applying distance-based clustering.

Key limitation: customers with missing Customer ID cannot be clustered (they have no stable identifier), and aggregation can hide within-customer variability (e.g., a customer who changes behaviour over time).

Step 2 — Feature construction (customer behaviour profiles)

We keep a **purchases-only** dataset by removing cancellations (Invoices starting with "C"), and non-sensical values (Quantity <= 0 or Price <= 0).

```
df_raw <- readr::read_csv(
  here("data", "raw", "online_retail_II(4).csv"),
  show_col_types = FALSE
)

df1 <- df_raw %>%
  mutate(
    InvoiceDate = dmy_hms(InvoiceDate),
    TotalPrice = Quantity * Price,
    is_cancel   = str_starts(Invoice, "C")
  )

df_purchases <- df1 %>%
  filter(!is_cancel, Quantity > 0, Price > 0) %>%
  filter(!is.na(InvoiceDate))

# Keep only customers with IDs (needed for customer-level aggregation)
df_purchases_id <- df_purchases %>% filter(!is.na(`Customer ID`))
```

```
glimpse(df_purchases_id)
```

Rows: 407,664

Columns: 10

```
$ Invoice      <chr> "489434", "489434", "489434", "489434", "489434", "48943~
$ StockCode   <chr> "85048", "79323P", "79323W", "22041", "21232", "22064", ~
$ Description  <chr> "15CM CHRISTMAS GLASS BALL 20 LIGHTS", "PINK CHERRY LIGH~
$ Quantity    <dbl> 12, 12, 12, 48, 24, 24, 24, 10, 12, 12, 24, 12, 10, 18, ~
$ InvoiceDate  <dtm> 2020-12-01 09:07:45, 2020-12-01 09:07:45, 2020-12-01 09~
$ Price       <dbl> 6.95, 6.75, 6.75, 2.10, 1.25, 1.65, 1.25, 5.95, 2.55, 3.~
$ `Customer ID` <dbl> 13085, 13085, 13085, 13085, 13085, 13085, 13085, 13085, ~
$ Country     <chr> "United Kingdom", "United Kingdom", "United Kingdom", "U~
$ TotalPrice   <dbl> 83.40, 81.00, 81.00, 100.80, 30.00, 39.60, 30.00, 59.50, ~
$ is_cancel    <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, ~
```

We create **one row per customer** and compute behaviour features:

- **Recency (days):** how many days since last purchase (relative to dataset end).
- **Frequency:** number of invoices (repeat buying).
- **Monetary value:** total historical spend.
- Additional structure: basket size/value, product variety.

```
cutoff_date <- max(df_purchases_id$InvoiceDate, na.rm = TRUE)
```

```
cust <- df_purchases_id %>%
```

```
  group_by(`Customer ID`) %>%
```

```
  summarise(
```

```
    n_invoices      = n_distinct(Invoice),           # frequency proxy
```

```
    n_items         = n(),                           # engagement proxy
```

```
    n_products      = n_distinct(StockCode),         # variety proxy
```

```
    total_spend     = sum(TotalPrice, na.rm = TRUE),  # monetary proxy
```

```
    avg_item_price  = mean(Price, na.rm = TRUE),
```

```
    avg_basket_value = total_spend / n_invoices,
```

```
    avg_items_per_invoice = n_items / n_invoices,
```

```
    last_purchase   = max(InvoiceDate, na.rm = TRUE),
```

```
    recency_days    = as.numeric(difftime(cutoff_date, last_purchase, units = "days")),
```

```
    active_span_days = as.numeric(difftime(max(InvoiceDate), min(InvoiceDate), units = "days")),
```

```
    .groups = "drop"
```

```
  ) %>%
```

```
  select(-last_purchase)
```

```
summary(cust)
```

Customer ID	n_invoices	n_items	n_products
Min. :12346	Min. : 1.000	Min. : 1.00	Min. : 1.00
1st Qu.:13882	1st Qu.: 1.000	1st Qu.: 18.00	1st Qu.: 17.00
Median :15350	Median : 2.000	Median : 44.00	Median : 38.00
Mean :15349	Mean : 4.456	Mean : 94.54	Mean : 63.65
3rd Qu.:16834	3rd Qu.: 5.000	3rd Qu.: 102.00	3rd Qu.: 79.25
Max. :18287	Max. :205.000	Max. :5570.00	Max. :1741.00

total_spend	avg_item_price	avg_basket_value
Min. : 2.95	Min. :1.513e-01	Min. : 2.95
1st Qu.: 307.99	1st Qu.:2.303e+00	1st Qu.: 182.09
Median : 706.02	Median :2.971e+00	Median : 287.37
Mean : 2048.24	Mean :9.862e+00	Mean : 378.32
3rd Qu.: 1723.14	3rd Qu.:3.882e+00	3rd Qu.: 423.58
Max. :349164.35	Max. :1.095e+04	Max. :11880.84

avg_items_per_invoice	recency_days	active_span_days
Min. : 1.000	Min. : 0.00	Min. : 0
1st Qu.: 9.871	1st Qu.: 20.00	1st Qu.: 0
Median : 17.667	Median : 48.96	Median : 99
Mean : 22.128	Mean : 83.37	Mean :126
3rd Qu.: 28.000	3rd Qu.:111.96	3rd Qu.:244
Max. :248.000	Max. :352.96	Max. :354

1.1 Skew handling (practical)

Spend-like features are often extremely right-skewed. For clustering, this can cause a few large values to dominate distances even after scaling. A common, transparent fix is `log1p()` ($\log(1+x)$) for strictly-positive, heavy-tailed features.

```
cust_feat <- cust %>%
  mutate(
    total_spend_log = log1p(total_spend),
    avg_basket_value_log = log1p(avg_basket_value),
    n_invoices_log = log1p(n_invoices),
    n_products_log = log1p(n_products),
    n_items_log = log1p(n_items)
  )

cust_feat %>%
```

```
select(`Customer ID`, total_spend, total_spend_log, n_invoices, n_invoices_log) %>%
slice_head(n = 5)
```

```
# A tibble: 5 x 5
  `Customer ID` total_spend total_spend_log n_invoices n_invoices_log
      <dbl>      <dbl>      <dbl>      <int>      <dbl>
1      12346       373.        5.92         11        2.48
2      12347      1323.        7.19          2        1.10
3      12348       222.        5.41          1        0.693
4      12349      2671.        7.89          3        1.39
5      12351       301.        5.71          1        0.693
```

##Step 3 — Scaling (why it matters)

Clustering here is **distance-based**. If features are on different scales (e.g., `total_spend` in thousands while `recency_days` is in tens), then **large-scale features dominate** Euclidean distance and determine clusters regardless of the others.

Method used: standardisation (z-scores) with `scale()`.

What would happen without scaling: clusters would mostly separate customers by the largest-range features (typically spend), and recency/frequency would contribute little.

```
# Choose a compact feature set for clustering
X <- cust_feat %>%
  transmute(
    recency_days,
    n_invoices_log,
    total_spend_log,
    avg_basket_value_log,
    n_products_log
  )

# Standardise (mean 0, sd 1)
X_scaled <- scale(X)

# sanity check: means ~0, sds ~1
apply(X_scaled, 2, mean) %>% round(3)
```

```
      recency_days      n_invoices_log      total_spend_log
              0              0              0
avg_basket_value_log      n_products_log
              0              0
```

```
apply(X_scaled, 2, sd) %>% round(3)
```

```
      recency_days      n_invoices_log      total_spend_log
      1            1            1
avg_basket_value_log      n_products_log
      1            1
```

##Step 4 — Apply clustering

1.2 4A — K-means (required)

1.2.1 Choosing k (avoid arbitrary decisions)

K-means requires a number of clusters k . There is no single “correct” value, but we can make an informed choice using:

- **Elbow / within-cluster sum of squares (WSS):** diminishing returns as k increases.
- **Silhouette:** how well-separated clusters are (higher is better), but we treat it as guidance, not a goal.

```
wss <- map_dbl(2:10, ~kmeans(X_scaled, centers = .x, nstart = 25)$tot.withinss)

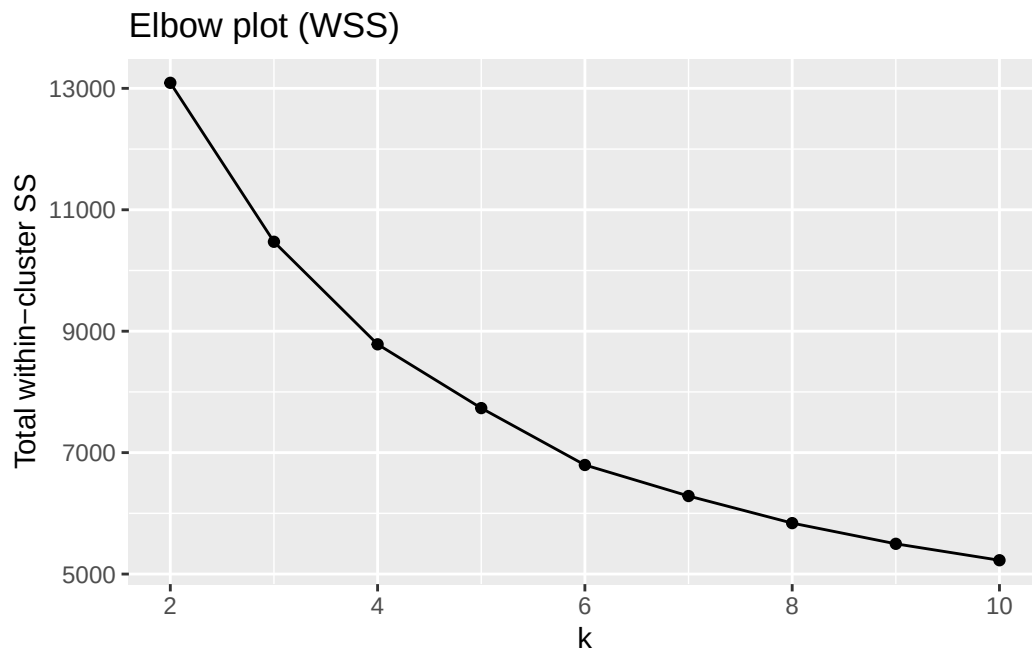
sil <- map_dbl(2:10, function(k){
  km <- kmeans(X_scaled, centers = k, nstart = 25)
  ss <- silhouette(km$cluster, dist(X_scaled))
  mean(ss[, 3])
})

k_diagnostics <- tibble(k = 2:10, wss = wss, silhouette = sil)
k_diagnostics
```

```
# A tibble: 9 x 3
      k    wss silhouette
  <int> <dbl>     <dbl>
1     2 13091.     0.337
2     3 10473.     0.275
3     4  8784.     0.263
4     5  7733.     0.249
5     6  6797.     0.261
6     7  6285.     0.244
```

7	8	5839.	0.232
8	9	5498.	0.224
9	10	5227.	0.216

```
ggplot(k_diagnostics, aes(k, wss)) +
  geom_line() + geom_point() +
  labs(title = "Elbow plot (WSS)", x = "k", y = "Total within-cluster SS")
```



```
ggplot(k_diagnostics, aes(k, silhouette)) +
  geom_line() + geom_point() +
  labs(title = "Average silhouette by k", x = "k", y = "Average silhouette")
```



Chosen k: I use $k = 4$ as a compromise between (1) clear diminishing returns in WSS after a small number of clusters and (2) a reasonable silhouette without producing tiny clusters that are hard to interpret.

(If your plots clearly suggest a different k , you can change it here and explain why.)

```
k <- 4
km <- kmeans(X_scaled, centers = k, nstart = 50)

cust_km <- cust_feat %>%
  mutate(cluster = factor(km$cluster))

cust_km %>% count(cluster) %>% arrange(desc(n))
```

```
# A tibble: 4 x 2
  cluster      n
  <fct>    <int>
1 4        1685
2 1         991
3 3         955
4 2         681
```


1.3 4B — Inspect cluster centres (interpretation-friendly)

K-means centres are in **scaled units**, so to interpret we compare **original-scale summaries** by cluster.

```
profiles <- cust_km %>%
  group_by(cluster) %>%
  summarise(
    n_customers = n(),
    recency_days_mean = mean(recency_days),
    n_invoices_mean = mean(n_invoices),
    total_spend_mean = mean(total_spend),
    avg_basket_value_mean = mean(avg_basket_value),
    n_products_mean = mean(n_products),
    .groups = "drop"
  ) %>%
  arrange(desc(n_customers))
```

profiles

```
# A tibble: 4 x 7
  cluster n_customers recency_days_mean n_invoices_mean total_spend_mean
  <fct>      <int>          <dbl>          <dbl>          <dbl>
1 4          1685          61.2           2.95          1122.
2 1           991          60.5           1.66           264.
3 3           955          26.4          12.3          6760.
4 2           681         251.           1.30           329.
# i 2 more variables: avg_basket_value_mean <dbl>, n_products_mean <dbl>
```

1.4 4C — Visualise clusters with PCA (2D view)

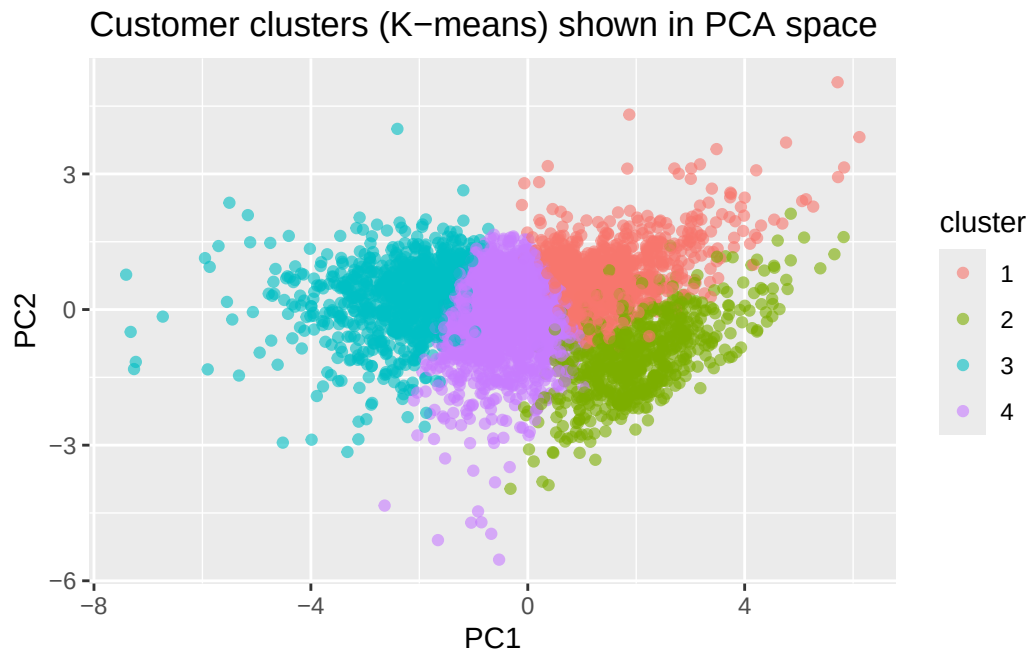
We reduce the (scaled) feature space to 2 principal components for a **visual diagnostic** (not proof of correctness).

```
pca <- prcomp(X_scaled, center = FALSE, scale. = FALSE)

pca_df <- as_tibble(pca$x[, 1:2]) %>%
  setNames(c("PC1", "PC2")) %>%
  mutate(cluster = cust_km$cluster)

ggplot(pca_df, aes(PC1, PC2, color = cluster)) +
```

```
geom_point(alpha = 0.6) +
labs(title = "Customer clusters (K-means) shown in PCA space")
```



Hierarchical clustering (comparison)

Hierarchical clustering can reveal whether the partitioning looks similar under a different assumption (agglomerative merging).

```
d <- dist(X_scaled)
hc <- hclust(d, method = "ward.D2")

# Cut to k clusters for a direct comparison
hc_clusters <- cutree(hc, k = k)

cust_hc <- cust_feat %>%
  mutate(hc_cluster = factor(hc_clusters))

cust_hc %>% count(hc_cluster) %>% arrange(desc(n))
```

```
# A tibble: 4 x 2
  hc_cluster      n
  <fct>          <int>
```

1	2	1944
2	1	1323
3	3	856
4	4	189

##Step 5 — Evaluate and interpret (critical discussion)

Use the profile table above to describe clusters in plain language. A typical pattern is:

- **Low recency (recent buyers) + higher frequency/value** → “active core customers”
- **High recency (haven’t bought recently) + low frequency/value** → “lapsed / one-off customers”
- **High basket value but low frequency** → “occasional big spenders”
- **High variety** → “browsers / diverse purchasers”

Write your final interpretation using your actual profiles output (cluster numbers are arbitrary; interpret by their statistics).

1.5 Are clusters balanced?

Discuss whether one cluster dominates (often there is a very large “low-activity” cluster). If extremely imbalanced, note that k-means can favour spherical clusters and may compress small but meaningful segments.

1.6 Sensitivity + limitations

- **Parameter sensitivity:** K-means depends on k and initialisation. We reduced this with `nstart = 50`, but different k can change the story.
- **Feature sensitivity:** clustering results are driven by feature choices. Adding/removing recency or log transforms can change geometry.
- **Scaling dependence:** without scaling, `total_spend` would dominate and clusters would mostly reflect spend alone.
- **Business meaning:** clusters may reflect data-collection artefacts (e.g., observation window, missing IDs) rather than real “types”.